# My Life Organized: Account Security, Intelligent Imports and Microsoft Store Roadmap

**Review edition:** October 15, 2026
**Application baseline:** MLO Studio v3.6, built from v3.5
**Website:** https://organizer.overvault.ca
**Deployment:** GitHub -> GitHub Actions -> Portainer -> Raspberry Pi-hosted FastAPI + SQLite database

> **The objective:** make MLO trustworthy enough for family and future public users, while keeping it pleasant to use, easy to maintain and free to download. This document distinguishes **implemented in v3.6**, **requires configuration**, and **planned future development**.

## 1. Executive decisions

| Decision | Product behavior |
|---|---|
| One identity per person | All users have an individual sign-in and account ID. Shared projects require explicit permissions. |
| Admin-controlled recovery | Administrators can reset member passwords and require a change at the next login. **Implemented in v3.6.** |
| Email-ready accounts | Accounts can store recovery email addresses and email-verification state. **Implemented in v3.6.** |
| Self-service recovery | A short-lived one-use email link can reset a password, but only after email verification and SMTP configuration. **Code included, SMTP configuration required.** |
| Session safety | Resetting a password revokes preexisting sessions. **Implemented in v3.6.** |
| Windows app | Publish a hosted Progressive Web App (PWA), not a second backend. Both browser and Windows app use the same production database. **Release planned, not submitted.** |
| Smart syllabus imports | PDF/DOCX/image -> extract -> confidence and provenance -> review -> save. **Planned.** |
| Smart bank imports | CSV/OFX/PDF -> normalize -> reconcile -> preview -> confirm. **Planned.** |

| Decision | Product behavior |
| --- | --- |
| Data ownership | All personal records must be user-scoped before wide public access. **Legacy academic/financial modules remain a gap.** |

# 2. What’s shipping in the v3.6 source package

## 2.1 Administrator account management

- Existing **Users** view keeps approve/revoke and enable/disable actions.
- New individual capability controls: create projects, share projects/spaces, connect calendars, upload project files, import tasks, manage notification devices.
- Enforcement is on the server for the protected actions, not just by hiding buttons. These flags do **not** grant access to older global finance/academic tables.
- Administrator can issue a temporary password of 12 or more characters, after re-entering the administrator’s own password. Temporary passwords are **never emailed automatically or displayed after submission**. Share them privately with the user.
- The affected user’s sessions are invalidated and the next sign-in requires a password change.
- Administrator can set/change a member’s recovery email after confirming their own password. The replacement is **unverified** until the user proves ownership.
- A basic security audit trail records password and permission changes. **This is not yet a full enterprise security log** and still needs operator-facing audit-reporting screens.
- Admin accounts cannot be disabled through the member controls. Administrator role grants and legacy-data isolation need further hardening before adding organization roles.

## 2.2 Account ownership and password changes

- Signup supports an optional email. Old users keep their accounts and may add email later.
- Signed-in users can open **Account security** to add a recovery email, request verification, or change their password.
- Changing an email requires the current password. Recovery never operates on unverified email addresses.
- Forgot-password returns the **same response** for unknown usernames and known usernames, reducing account enumeration.
- Reset links are random, single-use, stored as hashes, expire after 30 minutes, and are limited to one issuance per user/purpose every five minutes.
- Password changes revoke sessions. When v3.6 first deploys, users with pre-v3.6 session cookies will be asked to sign in again.
- Passwords remain salted PBKDF2 hashes, never plaintext database fields.

### 2.3 Fix to the REEF project-deletion panel

The previous panel used a near-white pink background and light text, overriding the theme. It now uses the same theme-aware surface as surrounding cards, contrasting readable text, a controlled red accent only for the destructive action, consistent spacing, focus outlines and mobile sizing. Chromium checks passed for all six themes in both modes (12 combinations) at three viewport widths. The two actual screenshots are packaged under `docs/preview/`.

### 2.4 Safer upgrade behavior

**Important behavior change:** `APP_PASSWORD` is now a bootstrap-only credential. Earlier versions silently reset the bootstrap administrator password from the environment variable on each restart. v3.6 will not overwrite an established administrator password during deployment. Keep your chosen password safe and test recovery before deploying publicly. An automatic one-time `pre-mlo-v3.6.sqlite.backup` snapshot is created before the schema upgrade. Back up the **entire** Docker `/data` volume yourself before upgrading.

## 3. How account email and self-service password recovery work

### Immediately usable without email delivery

1. Person selects **Forgot password** on sign-in.
2. If SMTP isn't configured, the page explains that account recovery needs an administrator.
3. Admin opens **Users -> Account controls -> Reset password**, enters a temporary password and confirms with their own password.
4. Admin privately communicates the temporary password; the user must choose a permanent one.
5. Existing sessions are revoked and other users' records remain untouched.

### With email delivery enabled

1. Set verified domain-based email sending, e.g. `MLO_SMTP_HOST`, `MLO_SMTP_USER`, `MLO_SMTP_PASSWORD`, `MLO_SMTP_FROM` and `MLO_SMTP_PORT` in **Portainer environment variables**, not GitHub.
2. Set the existing `MLO_PUBLIC_URL=https://organizer.overvault.ca` so links point to your real HTTPS website.
3. User opens **Account security** and adds a recovery email, confirming the change with their current password.
4. User clicks **Send verification email** and confirms through the one-hour, one-use link.
5. Only then can **Forgot password** send a password-reset link. Recovery links expire in 30 minutes; they are single-use.
6. The server stores only SHA-256 token digests. Uvicorn access logs are disabled to avoid exposing URL tokens. Cloudflare/reverse-proxy logs and third-party analytics must also avoid recording recovery URLs.

**Delivery and security prerequisites:** A reputable SMTP provider; SPF, DKIM and DMARC records for sender identity; outbound rate limiting at the edge; TLS; monitoring for failed email deliveries; and testing against Gmail/Outlook spam filters. Emails must not reveal temporary passwords. Don’t use a personal email password in Portainer if your provider supports a dedicated SMTP credential.

**Before a public launch, add:** passkeys/2FA for administrators, stronger login brute-force protection, account deletion/data export controls, explicit administrator session-management views, and recovery-token log redaction. Email verification is not a substitute for MFA.

# 4. Microsoft Store release: free and server-backed

## Architecture

Windows Store app -> installed PWA -> `https://organizer.overvault.ca` -> FastAPI app -> existing Docker `/data` volume. The application stores its authoritative user content on the Raspberry Pi-backed server. There is no separate per-device production database. Windows app and web browser show the same records when signed into the same account, subject to network and synchronization delays. Browser caches may store static UI assets; private documents and authenticated pages must not be publicly cached.

## Publishing checklist

1. Validate secure HTTPS, site accessibility, PWA manifest, 192/512 icons, service worker, launch behavior, page metadata and navigation on Windows.
2. Add a polished offline/error screen (without promising offline edits), keyboard accessibility, responsive UI, icon sizes, screenshots, support email and legal links.
3. Review and publish accurate **Privacy Policy** and **Terms of Service**; test account creation, sign-in, delete/request-delete and recovery.
4. Fix Google Search Console’s deceptive-pages warning and complete OAuth verification as applicable **before** inviting general users.
5. Register a Microsoft Store developer account using Microsoft’s new free onboarding process; choose Individual versus Company deliberately (changing later is not directly supported).
6. Reserve `My Life Organized` (or an available product name) in Partner Center.
7. Use PWABuilder with the live URL, fix its quality report, and generate Windows MSIX/ MSIXBundle packages using Partner Center identity values.
8. Test packaged app on Windows 10/11: login cookies, external OAuth handoff, deep links, file uploads, notification consent, updates and disconnect handling.
9. Submit a **free-to-download** listing with transparent online-account/network requirements, screenshots, accessibility notes, support contact and age rating.
10. After certification, stage a controlled public rollout, monitor crash/error rates, and back up/ restore the database routinely.

**Cost:** Current Microsoft guidance says new Individual and Company developer registration is free through the new flow. Free Store download is possible. This does **not** make hosting, domain, transactional email, AI APIs or other cloud usage free. Store certification is not guaranteed.

References: https://learn.microsoft.com/en-us/windows/apps/publish/partner-center/open-a-developer-account ; https://learn.microsoft.com/en-us/windows/apps/publish/publish-your-app/pwa/app-package-requirements

## 5. Intelligent course-outline / syllabus importer (next major revision)

**User journey:** Study -> Import syllabus -> upload file -> review recognized course facts and extracted due dates -> select existing/new course -> confirm -> optionally add reminders and schedule blocks.

**Formats, in order:** born-digital PDF and DOCX first; text from images/scanned PDFs only when necessary; CSV as optional explicit structured import. Parse text and tables, not just page-wide OCR. Retain the original file securely with a deletion/retention policy.

**Fields to propose:** course code/title, section, semester, instructor, email and office hours, class/lab times, lecture location, learning outcomes, textbooks, grade weights, assignment names, due dates/times, exams, reading schedule, holidays, participation requirements and grading policies. Show **unavailable/ambiguous** rather than inventing values.

**Required safety gates:**

- Resolve timezone, term/year, AM/PM, relative dates and whether dates have been announced.
- Show page/section source snippets and extraction confidence per field.
- Check weights for missing categories or totals that do not sum to 100%; warn rather than changing instructor grading rules.
- Offer row-by-row edit, skip, merge and duplicate detection based on course/term/event identity.
- Never directly create tasks, calendar events or reminders without an explicit confirmation screen.
- Use two-stage transactions and idempotency keys so retries do not create duplicates.
- Keep each imported course and its work private to the correct MLO account.
- Start with local parsing and simple heuristics; optional AI interpretation must be explicit, cost-limited, privacy-disclosed, and reviewed by a human.

**Acceptance test:** A representative set of syllabi (tables, multiple formats, scanned/poor-quality) imports with correct provenance, no hallucinated due dates, clear warnings for ambiguity, and zero duplicate rows on re-import.

## 6. Intelligent bank/e-statement importer (next major revision)

**User journey:** Money -> Import statement -> choose bank/account -> upload statement -> parse and reconcile -> preview categories/duplicates -> confirm -> see budget reports updated. Never request online banking username or password for file-based imports.

**Start with:** CSV, OFX/QFX where licensing/support permits, then born-digital PDFs, and scanned PDFs as a carefully controlled last resort. Banks change export formats; do not promise every statement will be parsed perfectly.

**Normalized transaction data:** statement source/bank, masked account identifier, currency, transaction date, posting date, merchant description, debit/credit direction, signed amount, running balance (if supplied), transaction ID, fees, transfers/refunds, category prediction and source provenance. Preserve exact original amounts and currency symbols. Reconcile opening balance + transactions = closing balance when available.

**Safeguards:**

- Require a user preview; let them correct merchants, dates, categories and account mappings before import.
- Detect duplicates with statement IDs or stable normalized transaction fingerprints; flag questionable matches for human review.
- Do not treat transfers between your own accounts as spending or double-count refunds.
- Flag ambiguous negative signs, reversed debit/credit columns and currency conversions.
- Ensure data is isolated per user. Restrict imports by capability where applicable.
- Put limits on file size, decompression, processing time and embedded objects. Scan attachment types, do not execute macros/scripts in user files.
- Encrypt or minimize raw financial-statement retention; maintain clear deletion/export procedures.
- Audit imported totals and rejected rows before committing.

**Acceptance test:** Reference statements from a variety of institutions, with no transaction duplication after re-import and successful reconciliation when source totals exist. Categorization may be imperfect, but numbers must never be silently guessed.

## 7. Additional code-review findings and production priorities

| Priority | Finding | Decision |
|---|---|---|
| P0 | Legacy school, finance, jobs and expenses still use some administrator-owned/shared data paths | Convert to strictly owner-scoped schemas before broad public distribution. Do not grant this access through simple role toggles. |

| Priority | Finding | Decision |
|---|---|---|
| P0 | Session cookies, password resets and project deletion deserve careful testing | Added account-security and theme regression tests. Add real deployment penetration testing and backup restore drill. |
| P0 | Existing signup approves accounts manually; email is optional | Keep manual approval while self-service recovery is optional and verified. Add abuse protections and account deactivation notifications later. |
| P1 | No trusted production SMTP integration yet | Configure SMTP and validate sender-domain DNS, deliverability, tokens and proxy log redaction. |
| P1 | Need audit-log viewer / user session admin controls | Store a basic audit trail now; build retention, export and administrator audit viewer next. |
| P1 | Existing Google and Microsoft calendar writes tested against mocks, not all real account types | Verify live create/edit/delete, refresh-token renewal, 401/429 backoff, duplicate updates and event recurrence on test calendars. |
| P1 | Background scheduler currently runs in the web container | Keep one application process or move scheduling to a dedicated worker before scaling horizontally. |
| P1 | Existing SQLite database is appropriate for modest personal usage but may bottleneck large multi-user edits | Benchmark concurrent writes; evaluate PostgreSQL only when indicated by load. |
| P1 | Deceptive-pages warning in Google Search Console may affect trust and OAuth | Investigate domain/subdomains, DNS, redirects, script dependencies, unknown owners and externally reachable pages before requesting review. |
| P1 | Cloudflare/Portainer secrets and personal data need recovery procedures | Keep encryption/session keys in protected Portainer settings and in a secure password manager; test complete `/data` restore. |
| P2 | True offline editing and conflict merge aren’t implemented | Keep Windows PWA server-backed. Do not present a static cache as offline synchronization. |
| P2 | Some complex legacy editing screens use older markup | Continue progressive component replacement and contrast/ accessibility checks. |

## 8. Recommended delivery phases

**Phase A, now:** deploy v3.6 to a clone/staging database; validate admin temporary-password resets, email fields, roles and permissions, one-use tokens, account login, projects and visual contrast. Take a complete production `/data` volume backup. Then deploy to the existing stack after successful tests.

**Phase B, before public beta:** configure and test SMTP, DNS for mail deliverability, log redaction, administrator MFA, login throttling, security incident process, full tenant isolation for legacy data, explicit account deletion and production monitoring.

**Phase C, intelligent imports:** ship the syllabus importer with review/confirmation, then the statement importer with transaction reconciliation and duplicate prevention. Make reliability and safe data handling acceptance gates.

**Phase D, Microsoft Store:** package the validated hosted PWA, finish OAuth/security review, submit a free listing and monitor actual usage. Keep the server as the single source of truth.

## 9. v3.6 acceptance results and boundaries

**Locally verified:** Python compilation, template rendering, account/admin auth flows, account isolation, permission enforcement, admin temporary password, forced password changes, session revocation, verified recovery email, single-use reset tokens, migrations, project deletion, calendar routes and visual contrast in 12 theme combinations.

**Not claimed:** real SMTP delivery, live Google/Microsoft calendar APIs, Microsoft Store acceptance, production Raspberry Pi/Portainer deployment, regulated financial-data compliance, complete tenant isolation of legacy finance/academic data, or automatic syllabus/bank-statement extraction. Those still require deployment and/or future development.

**Upgrade note:** This release requires existing users to sign in again because it introduces explicit session-version tracking. The existing `APP_PASSWORD` no longer silently resets an established administrator password after reboot. Keep the bootstrap admin credential secure, and configure recovery before distributing the app widely.